1.Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.
Answer:

```
CREATE TABLE Restaurant(
        id INT PRIMARY KEY,
        name VARCHAR() NOT NULL,
        location VARCHAR() NOT NULL,
        cuisine VARCHAR()
);
```

2.Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).
Answer:

```
CREATE TABLE FoodOrder(
        order_id INT PRIMARY KEY,
        restaurant_id INT,
        user_id PRIMARY KEY,
        order_total DECIMAL(),
        FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)
);
```

3.Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.
Answer:

```
INSERT INTO SpotifyUser(‘user_id’,’username’,’email’,’subscription_type’) VALUES
(1, 'melodic_mind', 'melodic@gmail.com', 'Premium'),
(2, 'beat_drop', 'beatdrop@yahoo.com', 'Free'),
(3, 'audio_phile', 'audio@outlook.com', 'Premium'),
(4, 'groove_master', 'groove@gmail.com', 'Family'),
(5, 'sound_wave', 'wave@icloud.com', 'Free'),
(6, 'rhythm_vibes', 'rhythm@gmail.com', 'Student');
```

4.Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.
Answer:

```
CREATE VIEW TopSpendersView AS
SELECT username, order_total
FROM FoodOrder
WHERE order_total > 1000;
```

5.Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending.
Answer:

```
SELECT username, order_total FROM TopSpendersView
WHERE order_total > 2000
ORDER BY order_total DESC;
```